

Research Notes on CycleGAN

CycleGAN

>> Section 1

Choice of the strategy set In the most generic version of the GAN game described above, the strategy set for the discriminator contains all Markov kernels $\mu_D : \Omega \rightarrow \mathcal{P}[0, 1]$, and the strategy set for the generator contains arbitrary probability distributions μ_G on Ω . However, as shown below, the optimal discriminator strategy against any μ_G is deterministic, so there is no loss of generality in restricting the discriminator's strategies to deterministic functions $D : \Omega \rightarrow [0, 1]$. In most applications, D is a deep neural network function. As for the generator, while μ_G could theoretically be any computable probability distribution, in practice, it is usually implemented as a pushforward: $\mu_G = \mu_Z \circ G^{-1}$. That is, start with a random variable $z \sim \mu_Z$, where μ_Z is a probability distribution that is easy to compute (such as the uniform distribution, or the Gaussian distribution), then define a function $G : \Omega_Z \rightarrow \Omega$. Then the distribution μ_G is the distribution of $G(z)$. Consequently, the generator's strategy is usually defined as just G , leaving $z \sim \mu_Z$ implicit. In this formalism, the GAN game objective is $L(G, D) := \mathbb{E}_{x \sim \mu_{\text{ref}}} [\ln D(x)] + \mathbb{E}_{z \sim \mu_Z} [\ln(1 - D(G(z)))]$.

>> Section 2

StyleGAN-2 StyleGAN-2 improves upon StyleGAN-1, by using the style latent vector to transform the convolution layer's weights instead, thus solving the "blob" problem. This was updated by the StyleGAN-2-ADA ("ADA" stands for "adaptive"), which uses invertible data augmentation as described above. It also tunes the amount of data augmentation applied by starting at zero, and gradually increasing it until an "overfitting heuristic" reaches a target level, thus the name "adaptive".

>> Section 3

Concerns have been raised about the potential use of GAN-based human image synthesis for sinister purposes, e.g., to produce fake, possibly incriminating, photographs and videos. GANs can be used to generate unique, realistic profile photos of people who do not exist, in order to automate creation of fake social media profiles. In 2019 the state of California considered and passed on October

3, 2019, the bill AB-602, which bans the use of human image synthesis technologies to make fake pornography without the consent of the people depicted, and bill AB-730, which prohibits distribution of manipulated videos of a political candidate within 60 days of an election. Both bills were authored by Assembly member Marc Berman and signed by Governor Gavin Newsom. The laws went into effect in 2020. DARPA's Media Forensics program studies ways to counteract fake media, including fake media produced using GANs.

>> Section 4

Invertible data augmentation When there is insufficient training data, the reference distribution μ_{ref} cannot be well-approximated by the empirical distribution given by the training dataset. In such cases, data augmentation can be applied, to allow training GAN on smaller datasets. Naïve data augmentation, however, brings its problems. Consider the original GAN game, slightly reformulated as follows: $\min_D L(D, \mu_G) = -\mathbb{E}_{x \sim \mu_{\text{ref}}} [\ln D(x)] - \mathbb{E}_{x \sim \mu_G} [\ln(1 - D(x))]$. $\min_G L(G, D, \mu_G) = -\mathbb{E}_{x \sim \mu_G} [\ln(1 - D(x))]$. Now we use data augmentation by randomly sampling semantic-preserving transforms $T : \Omega \rightarrow \Omega$ and applying them to the dataset, to obtain the reformulated GAN game: $\min_D L(D, \mu_G) = -\mathbb{E}_{x \sim \mu_{\text{ref}}} [\ln D(T(x))] - \mathbb{E}_{x \sim \mu_G} [\ln(1 - D(x))]$. $\min_G L(G, D, \mu_G) = -\mathbb{E}_{x \sim \mu_G} [\ln(1 - D(x))]$. This is equivalent to a GAN game with a different distribution μ_{ref}' , sampled by $T(x)$, with $x \sim \mu_{\text{ref}}$, $T \sim \mu_{\text{trans}}$. For example, if μ_{ref} is the distribution of images in ImageNet, and μ_{trans} samples identity-transform with probability 0.5, and horizontal-reflection with probability 0.5, then μ_{ref}' is the distribution of images in ImageNet and horizontally-reflected ImageNet, combined. The result of such training would be a generator that mimics μ_{ref}' .

Research Notes on CycleGAN

CycleGAN

example, it would generate images that look like they are randomly cropped, if the data augmentation uses random cropping. The solution is to apply data augmentation to both generated and real images: $\min_{D, G} \mathbb{E}_{x \sim \mu_{\text{ref}}} [D(x)] - \mathbb{E}_{x \sim \mu_G} [D(x)] + \min_{G, D} \mathbb{E}_{x \sim \mu_G} [D(x)] - \mathbb{E}_{x \sim \mu_{\text{ref}}} [D(x)]$
$$\{\begin{cases} \min_{D, G} \mathbb{E}_{x \sim \mu_{\text{ref}}} [D(x)] - \mathbb{E}_{x \sim \mu_G} [D(x)] \\ \min_{G, D} \mathbb{E}_{x \sim \mu_G} [D(x)] - \mathbb{E}_{x \sim \mu_{\text{ref}}} [D(x)] \end{cases}$$
 The authors demonstrated high-quality generation using just 100-picture-large datasets. The StyleGAN-2-ADA paper points out a further point on data augmentation: it must be invertible. Continue with the example of generating ImageNet pictures. If the data augmentation is "randomly rotate the picture by 0, 90, 180, 270 degrees with equal probability", then there is no way for the generator to know which is the true orientation: Consider two generators G, G' , such that for any latent z , the generated image $G(z)$ is a 90-degree rotation of $G'(z)$. They would have exactly the same expected loss, and so neither is preferred over the other. The solution is to only use invertible data augmentation: instead of "randomly rotate the picture by 0, 90, 180, 270 degrees with equal probability", use "randomly rotate the picture by 90, 180, 270 degrees with 0.1 probability, and keep the picture as it is with 0.7 probability". This way, the generator is still rewarded to keep images oriented the same way as un-augmented ImageNet pictures. Abstractly, the effect of randomly sampling transformations $T: \Omega \rightarrow \Omega$ from the distribution μ_{trans} is to define a Markov kernel $K_{\text{trans}}: \Omega \rightarrow \mathcal{P}(\Omega)$. Then, the data-augmented GAN game pushes the generator to find some $\mu^G \in \mathcal{P}(\Omega)$, such that $K_{\text{trans}} * \mu_{\text{ref}} = K_{\text{trans}} * \mu^G$ where $*$ is the Markov kernel convolution. A data-augmentation method is defined to be invertible if its Markov kernel K_{trans} satisfies $K_{\text{trans}} * \mu = K_{\text{trans}} * \mu' \implies \mu = \mu' \forall \mu, \mu' \in \mathcal{P}(\Omega)$. Immediately by definition, we see that composing multiple invertible data-augmentation methods results in yet another invertible method. Also by definition, if the data-augmentation method is invertible, then using it

in a GAN game does not change the optimal strategy μ^G for the generator, which is still μ_{ref} . There are two prototypical examples of invertible Markov kernels: Discrete case: Invertible stochastic matrices, when Ω is finite. For example, if $\Omega = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ is the set of four images of an arrow, pointing in 4 directions, and the data augmentation is "randomly rotate the picture by 90, 180, 270 degrees with probability p , and keep the picture as it is with probability $(1 - 3p)$ ", then the Markov kernel K_{trans} can be represented as a stochastic matrix: $[K_{\text{trans}}] = \begin{bmatrix} (1-3p)p & p & p & p \\ p & (1-3p)p & p & p \\ p & p & (1-3p)p & p \\ p & p & p & (1-3p)p \end{bmatrix}$ and K_{trans} is an invertible kernel iff $[K_{\text{trans}}]$ is an invertible matrix, that is, $p \neq 1/4$. Continuous case: The gaussian kernel, when $\Omega = \mathbb{R}^n$ for some $n \geq 1$. For example, if $\Omega = \mathbb{R}^{256 \times 256}$ is the space of 256×256 images, and the data-augmentation method is "generate a gaussian noise $z \sim \mathcal{N}(0, I_{256 \times 256})$, then add ϵz to the image", then K_{trans} is just convolution by the density function of $\mathcal{N}(0, \epsilon^2 I_{256 \times 256})$. This is invertible, because convolution by a gaussian is just convolution by the heat kernel, so given any $\mu \in \mathcal{P}(\mathbb{R}^n)$, the convolved distribution $K_{\text{trans}} * \mu$ can be obtained by heating up $\mathbb{R}^n$ precisely according to μ , then wait for time $\epsilon^2/4$. With that, we can recover μ by running the heat equation backwards in time for $\epsilon^2/4$. More examples of invertible data augmentations are found in the paper.